02 06 Tag 3 Live-Angriffssimulation

1. Vorbereitung: Baseline-Check

Durchgeführte Schritte:
- Kibana Suricata-Dashboard geöffnet
- Ruhigen Zustand (Baseline) überprüft

Beobachtung:
- Aktuelle Event-Rate: 30 Events pro Minute

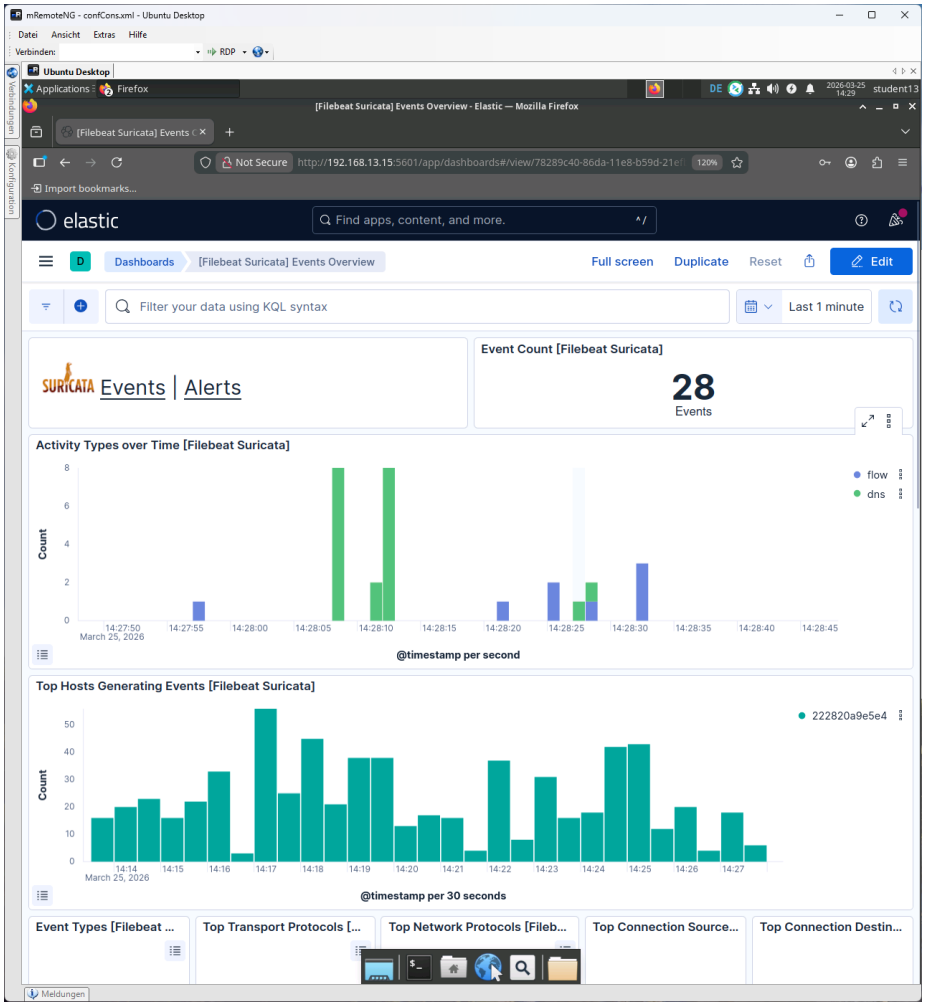


Figure 1: 20260325_142954_202525_mRemoteNG_-*confCons.xml*-_Ubuntu_Desktop

2. Angriff 1: Netzwerk-Portscan durchführen

Durchgeführte Schritte:
- Terminal auf dem Security-Server geöffnet cCc - Command ausgeführt: nmap -sV 192.168.13.1

Beobachtungen während des Scans:
- Zu Kibana gewechselt
- Neue Events sichtbar? Eventrate steigt für eine Minute auf 2000 Events an.

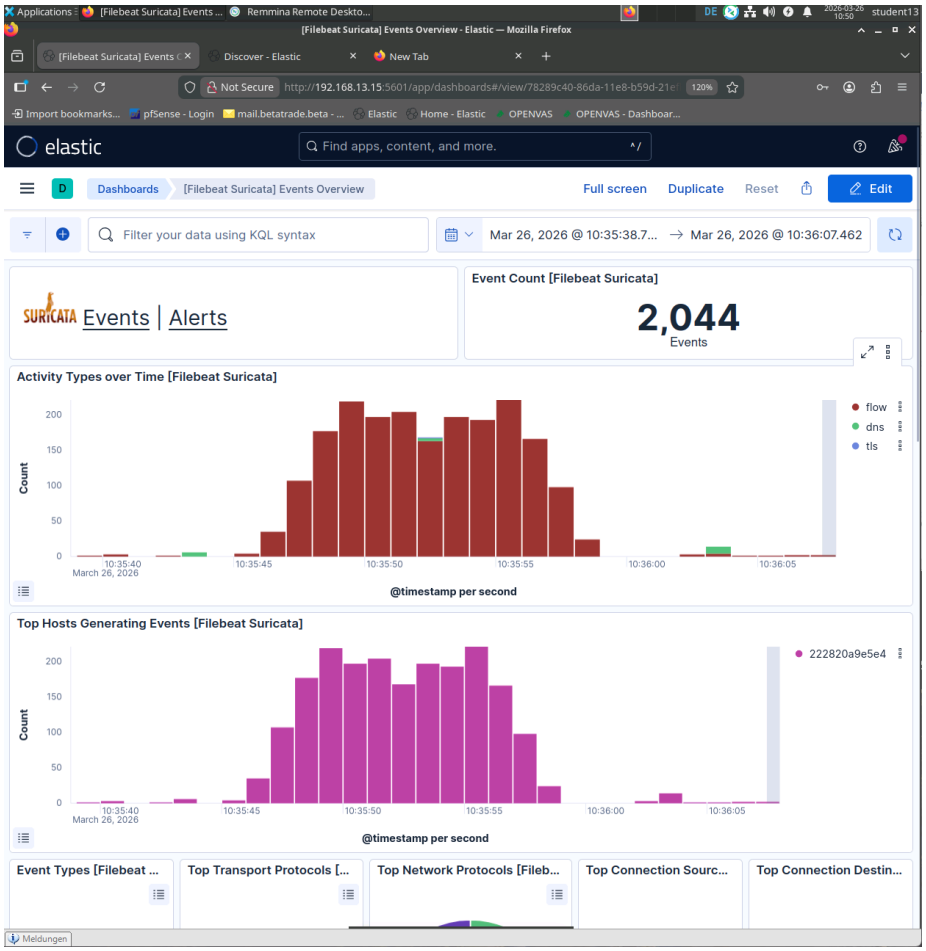
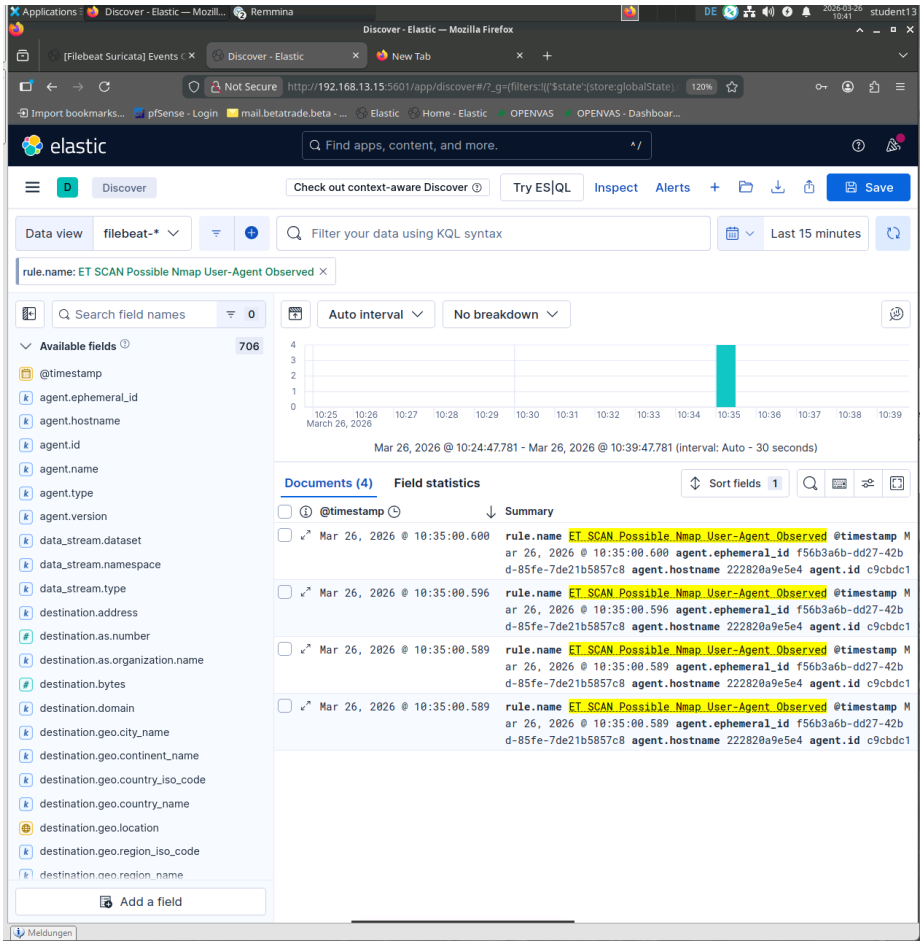


Figure 2: Pasted image 20260326105138

Dokumentation:
- Wurde der Scan erkannt? Ja - Anzahl der generierten Events: 2000 - Event-

Typen: 

---

3. Alert-Kategorien einordnen

Erklärung der Alert-Klassen (basierend auf Suricata + Kibana-Setup):

- **INFO:** Reine Informations-Events ohne unmittelbare Bedrohung. Dienen der Protokollierung normaler Aktivitäten oder niedrigstufiger Beobachtungen (niedrige Priorität, oft nur für Monitoring).

- **POLICY:** Verletzungen von definierten Sicherheitsrichtlinien (Policy Violations). Beispiele: Unerlaubte Protokolle, nicht-konforme Anwendungen oder Zugriffe, die gegen Unternehmensvorgaben verstoßen.

- **SCAN:** Reconnaissance-Aktivitäten (Aufklärung). Typisch für Portscans,

Host-Discovery oder andere Vorbereitungen eines Angriffs (in Suricata oft `classtype: attempted-recon`).

- **ATTACK:** Direkte Angriffe oder Exploits. Hohe Priorität – z. B. Exploit-Versuche, Malware-Kommunikation oder Brute-Force-Attacken.

Klassifizierung von bereits bekannten oder recherchierten Alerts:
- Der Portscan aus Aufgabe 2 → **SCAN** (Reconnaissance).
- Beispiel aus Recherche (Nmap-XMAS-Scan): **SCAN** (attempted-recon).
- Typischer POLICY-Alert: Verbotener Zugriff auf einen internen Port → **POLICY**.
- Typischer ATTACK-Alert: Exploit-Versuch (z. B. SQL-Injection-Payload) → **ATTACK**.
- INFO-Beispiel: Normaler HTTP-Request ohne Bedrohung → **INFO**.

---

4. Recherche: Wie funktionieren Suricata-Rules?

Ausgewählte Regel (Beispiel einer Portscan-Erkennungsregel):
Ich habe folgende gängige Regel ausgewählt (ähnlich zu Community-Regeln für SYN-Portscans, wie sie häufig in Suricata-Setups verwendet werden):

suricata alert tcp any any -> $HOME_NET any (msg:"Possible Port Scan Detected"; flags:S; threshold:type threshold,track by_src,count 5,seconds 10; sid:9000003; rev:3;)

Was matched die Regel?

> [!info] Regel-Trigger TCP-Pakete mit gesetztem **SYN-Flag** (`flags:S`), die **mehr als 5-mal innerhalb von 10 Sekunden** von derselben Quell-IP (`track by_src`) kommen.

Der **Threshold** erkennt schnelle Scans – z. B. `nmap -sV`, das intern SYN-Pakete verwendet.

Auf welcher Ebene greift die Regel?

Ebene	Details
Header-Ebene	TCP-Header (Flags, Quell-/Ziel-Ports via `$HOME_NET`-Variable)
Kein Payload	Keine Prüfung des Dateninhalts (`content:` oder `dsize:` fehlt)
Verhaltensebene	Flow-Tracking + Threshold (Rate-Limiting)

Warum kann das ein False Positive sein?

> [!warning] False Positive Risiko Legitimer Traffic kann mehrere SYN-Pakete erzeugen, z. B.:
>
> - Load Balancer
> - Web-Crawler
> - Viele parallele Verbindungen einer App
> - Backup-Prozesse

Der Threshold ist **nicht 100 % spezifisch** für böswillige Scans und löst bei hohem normalem Verkehr aus.

> [!tip] Lösung
>
> - Whitelisting von bekannten IPs
> - Feinabstimmung des Thresholds

ELK-Stack – Überblick

Was ist der ELK-Stack?

Der **ELK-Stack** (Elasticsearch, Logstash, Kibana – heute meist *Elastic Stack* genannt) ist eine **Open-Source-Plattform** zur zentralen Sammlung, Speicherung, Suche, Analyse und Visualisierung großer Mengen von Log- und Event-Daten.

Allgemeiner Einsatz: Zentrales Log-Management – Logs aus vielen Quellen werden aggregiert, indiziert und in Echtzeit durchsuchbar gemacht. Ermöglicht schnelle Abfragen, Korrelation von Events und interaktive Dashboards.

Einsatzbereiche

Bereich	Beschreibung
IT-Betrieb (Operations)	System-Monitoring, Performance-Analyse, Fehlerbehebung
Security (Cybersecurity)	SIEM für Threat Detection, Incident Response, Compliance
Business / DevOps	Application Monitoring, User-Behavior-Analyse, Business Intelligence

---

Komponenten des ELK-Stacks

```
Filebeat        Elasticsearch          Kibana
Logstash          (Speicher +            (Visualisier-
(Shipper)          Index)                 ung)
```

- **Elasticsearch** – Zentrale Such- und Analysedatenbank. Speichert und indiziert alle Logs als JSON-Dokumente, ermöglicht ultraschnelle Volltext- und aggregierte Suchen.
- **Logstash** – Verarbeitet und transformiert Logs (Parsing, Filter).
- **Filebeat** (Beats-Familie) – Leichtgewichtiger Shipper, der z. B. Suricata-Logdateien liest und direkt an Elasticsearch sendet (sehr effizient).
- **Kibana** – Die Visualisierungs- und Dashboard-Oberfläche. Erstellt Graphen, Suchen, Alerts und das Suricata-Dashboard.

Suricata im ELK-Stack

Was macht Suricata selbst?

Suricata ist ein hochperformantes **Network Intrusion Detection/Prevention System (NIDS/NIPS)**. Es analysiert Echtzeit-Netzwerk-Traffic paketweise anhand von Regeln und generiert Alerts bei Verdacht auf Scans oder Angriffe.

Welche Daten liefert Suricata an den ELK-Stack?

Suricata schreibt alle Events im **EVE-JSON-Format** (`eve.json`):

- Alerts (mit Nachricht, Classtype, SID)
- Flow-Informationen
- HTTP- / DNS- / TLS-Logs
- Datei-Extraktionen

Warum ist Suricata kein Teil des ELK-Stacks?

> [!note] Arbeitsteilung
>
> - **ELK** = Log-Management & Analytics (Speichern + Visualisieren)
> - **Suricata** = Detector (Traffic-Inspektion)

Zusammen ergibt das ein vollwertiges SIEM:

```
Netzwerk-Traffic
```

```
         eve.json
Suricata            Filebeat     Elasticsearch
(Detect)

                                                Kibana
                                               (Dashboard
                                                & Alerts)
```

Ohne…	Problem
Ohne Suricata	ELK hat keine netzwerkbasierten Security-Events
Ohne ELK	Suricata-Alerts sind schwer skalierbar zu analysieren